

National University of Computer & Emerging Sciences

Computer Science Department

Course Code: CL-2006	Course: Operating System Lab
Instructor: Yasir Arfat	Semester: Spring 2025

Lab Manual – 07

Inter-Process Communication (IPC) – Part 2 Message Queues & Shared Memory

Table of Contents

Table of Contents.....	2
Objectives	3
Recap: IPC Mechanisms.....	4
Message Queues.....	5
What is a Message Queue?	5
Key Concepts	5
System V Message Queue API	6
Example 1: Message Queue Sender.....	6
Example 2: Message Queue Receiver	7
Shared Memory	10
What is Shared Memory?	10
How Shared Memory Works (Step by Step)	11
System V Shared Memory API	11
Example 3: Shared Memory Server (Writer).....	11
Example 4: Shared Memory Client (Reader).....	12
Choosing the Right IPC Mechanism.....	15
Performance Comparison.....	15
System Calls – Quick Reference.....	16
Message Queue System Calls	16
Shared Memory System Calls	16
Useful Shell Commands	16
Lab Tasks	17
Task 1: Message Queue – Student Record System	17
Task 2: Message Queue with Priority	17
Task 3: Shared Memory – Chat Application.....	17
Task 4: Shared Memory Counter – Race Condition Demo	17
Task 5: IPC Cleanup Script (Bonus)	18

Objectives

This lab continues the IPC series from Lab 06. In Lab 06, you learned file descriptors, unnamed pipes, and named pipes (FIFO). Now you will explore two more powerful IPC mechanisms: message queues and shared memory. Upon completion, students will be able to:

- Understand how System V message queues work and when to use them
- Use `ftok()` to generate unique IPC keys for message queues and shared memory
- Send and receive structured messages using `msgsnd()` and `msgrcv()`
- Select messages by type for priority-based communication
- Understand how shared memory works at the kernel level (virtual → physical mapping)
- Create, attach, read/write, detach, and remove shared memory segments
- Use `shmget()`, `shmat()`, `shmdt()`, and `shmctl()` system calls
- Understand the race condition problem with shared memory (preparation for semaphores)
- Compare all IPC mechanisms covered so far and choose the right one for a given scenario

Recap: IPC Mechanisms

In Lab 06, we covered file descriptors (the foundation of all Linux I/O), unnamed pipes (for parent–child communication), and named pipes/FIFOs (for unrelated processes). Both pipes transfer raw bytes with no message boundaries and have built-in synchronization.

Now we move to two mechanisms that solve different problems:

- **Message Queues** — transfer structured, typed messages (not just raw bytes). Messages have boundaries and can be selected by type.
- **Shared Memory** — the fastest IPC method. Multiple processes directly access the same physical memory. No data copying through the kernel.

Feature	Pipes (Lab 06)	Message Queues (this lab)	Shared Memory (this lab)
Data format	Raw byte stream	Structured messages with type	Raw memory (any data)
Message boundaries	No	Yes — each message is discrete	No — you manage the layout
Speed	Fast	Medium	Fastest (zero-copy)
Synchronization	Built-in (blocking read/write)	Built-in (blocking receive)	NONE — you must handle it
Persistence	Until pipe closes	Until explicitly deleted	Until explicitly deleted
Direction	Unidirectional	Bidirectional (via types)	Bidirectional
File descriptors?	Yes — pipe FDs	No — uses queue IDs	No — uses segment IDs + pointers

Message Queues

What is a Message Queue?

A message queue is a kernel-managed buffer that holds discrete messages. Unlike pipes which transfer a continuous stream of bytes, message queues preserve message boundaries — each message you send is received as a complete unit on the other side.

Think of it like an email inbox: each email is a separate message with its own subject (type) and body (data). You can read emails in order, or you can search for a specific subject. Similarly, processes can send messages with different types and receivers can choose which type to read first.

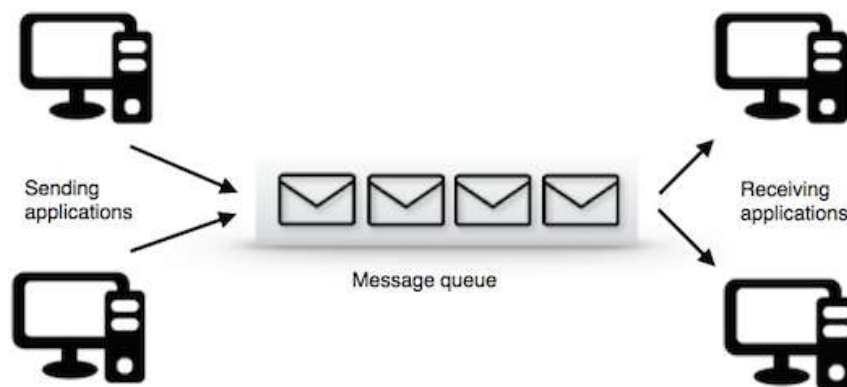


Figure 1: Message queue – Multiple sending applications place messages into a shared queue. Receiving applications read messages from the queue.

OS CONCEPT

Message queues are managed entirely by the kernel. When a process sends a message, the data is copied from the sender's address space into a kernel buffer. When another process receives the message, the data is copied from the kernel buffer into the receiver's address space. This means two data copies happen (sender→kernel→receiver), making message queues slower than shared memory but safer — processes never directly access each other's memory.

Key Concepts

IPC Key: Every message queue (and shared memory segment) needs a unique key so that different processes can find the same queue. The `ftok()` function generates a key from a file path and a project identifier:

```
key_t key = ftok("somefile.c", 'b');
// Generates a unique key based on the file's inode and the character 'b'
// Both sender and receiver must use the SAME file path and character
```

Message Structure: Every message must start with a long integer field (mtype) that identifies the message type. The rest of the structure is your data:

```
struct msgbuf {
    long mtype;           // Message type (MUST be > 0)
    char msgtxt[200];     // Your data (can be any type/size)
};
```

NOTE

The mtype field is critical: it MUST be a positive integer (> 0). When receiving, you can specify which type to read. If mtype = 0 in msgrcv(), it reads the first message regardless of type. If mtype = 3, it reads only messages of type 3. This enables priority-based communication.

System V Message Queue API

System Call	Header	Purpose
ftok(path, proj_id)	<sys/ipc.h>	Generate a unique IPC key from a file path and project ID
msgget(key, flags)	<sys/msg.h>	Create or open a message queue (returns queue ID)
msgsnd(qid, msg, size, flags)	<sys/msg.h>	Send a message to the queue
msgrcv(qid, buf, size, type, flags)	<sys/msg.h>	Receive a message from the queue (blocks if empty)
msgctl(qid, cmd, buf)	<sys/msg.h>	Control operations: IPC_RMID to delete the queue

Example 1: Message Queue Sender

This program creates a message queue, prompts the user for input, and sends each line as a message. When the user presses Ctrl+D (end of input), the queue is deleted.

message_send.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>    // ftok()
#include <sys/types.h>
#include <sys/msg.h>    // msgget(), msgsnd(), msgrcv(), msgctl()

// Message structure: mtype + data
struct msgbuf {
```

```

    long mtype;           // Message type (must be > 0)
    char msgtxt[200];     // Message payload
};

int main(void) {
    struct msgbuf msg;
    int msgid;
    key_t key;

    // Step 1: Generate a unique key
    // Both sender and receiver MUST use the same file + project ID
    key = ftok("message_send.c", 'b');
    if (key == -1) { perror("ftok"); exit(1); }

    // Step 2: Create the message queue (or open if it already exists)
    // 0644 = owner(rw), group(r), others(r)
    msgid = msgget(key, 0644 | IPC_CREAT);
    if (msgid == -1) { perror("msgget"); exit(1); }

    printf("message_send [INFO] Queue ID: %d\n", msgid);
    printf("message_send [PROMPT] Enter messages (Ctrl+D to stop):\n");

    // Step 3: Send messages in a loop
    msg.mtype = 1;        // All messages are type 1
    while (fgets(msg.msgtxt, 200, stdin)) {
        if (msgsnd(msgid, &msg, sizeof(msg), 0) == -1) {
            perror("msgsnd");
            exit(1);
        }
    }

    // Step 4: Clean up – delete the queue
    msgctl(msgid, IPC_RMID, NULL);
    printf("message_send [INFO] Queue deleted.\n");
    return 0;
}

```

Example 2: Message Queue Receiver

message_receiver.c

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/types.h>
#include <sys/msg.h>

```

```

struct msgbuf {
    long mtype;
    char msgtxt[200];
};

int main(void) {
    struct msgbuf msg;
    int msgid;
    key_t key;

    // Use the SAME key as the sender
    key = ftok("message_send.c", 'b');
    if (key == -1) { perror("ftok"); exit(1); }

    // Open the EXISTING queue (no IPC_CREAT – sender must create first)
    msgid = msgget(key, 0644);
    if (msgid == -1) { perror("msgget"); exit(1); }

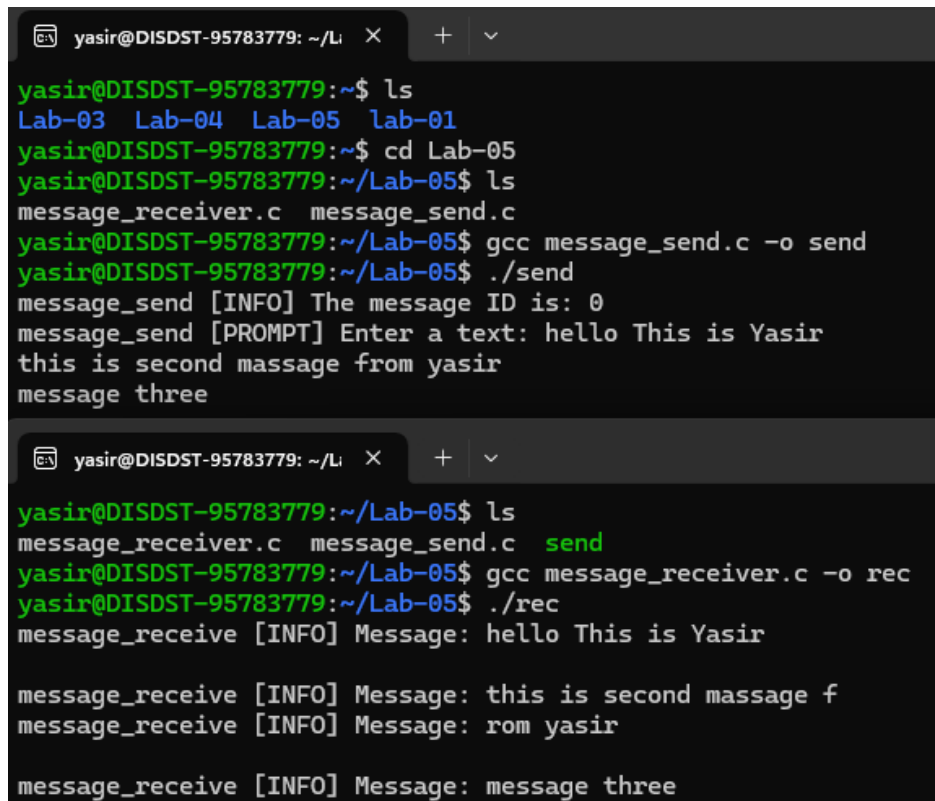
    printf("message_receive [INFO] Connected to Queue ID:%d\n", msgid);

    // Receive messages in an infinite loop
    // msgrcv with type=1 reads only type-1 messages
    // It BLOCKS if the queue is empty (built-in synchronization!)
    for (;;) {
        if (msgrcv(msgid, &msg, sizeof(msg), 1, 0) == -1) {
            perror("msgrcv");
            exit(1);
        }
        printf("message_receive [INFO] Message: %s", msg.msgtxt);
    }
    return 0;
}

```

Compile and run in two separate terminals:

# Terminal 1 (Sender)	# Terminal 2 (Receiver)
\$ gcc message_send.c -o send	\$ gcc message_receiver.c -o rec
\$./send	\$./rec
message_send [INFO] Queue ID: 0	message_receive [INFO] Connected...
Enter messages (Ctrl+D to stop):	
hello This is Yasir	message_receive [INFO] Message: hello This is
Yasir	
this is second message	message_receive [INFO] Message: this is
second message	



```

yasir@DISDST-95783779: ~/Li X + v
yasir@DISDST-95783779:~$ ls
Lab-03 Lab-04 Lab-05 lab-01
yasir@DISDST-95783779:~$ cd Lab-05
yasir@DISDST-95783779:~/Lab-05$ ls
message_receiver.c message_send.c
yasir@DISDST-95783779:~/Lab-05$ gcc message_send.c -o send
yasir@DISDST-95783779:~/Lab-05$ ./send
message_send [INFO] The message ID is: 0
message_send [PROMPT] Enter a text: hello This is Yasir
this is second massage from yasir
message three

yasir@DISDST-95783779: ~/Li X + v
yasir@DISDST-95783779:~/Lab-05$ ls
message_receiver.c message_send.c send
yasir@DISDST-95783779:~/Lab-05$ gcc message_receiver.c -o rec
yasir@DISDST-95783779:~/Lab-05$ ./rec
message_receive [INFO] Message: hello This is Yasir

message_receive [INFO] Message: this is second message f
message_receive [INFO] Message: rom yasir

message_receive [INFO] Message: message three

```

Figure 2: Message queue in action – Sender types messages (top), receiver displays them (bottom). Messages are received in the same order they were sent.

WARNING

The sender must be started FIRST (or at least must create the queue first with `IPC_CREAT`). If the receiver tries to open a non-existent queue, `msgget()` will fail. Also, if the sender ends without deleting the queue (crashes or misses `msgctl()`), the queue persists in the kernel. Use the command `ipcs -q` to list orphaned queues and `ipcrm -q <msgid>` to remove them.

TIP

To see all active IPC resources on your system, run: `ipcs`. This shows message queues (-q), shared memory (-m), and semaphores (-s). Very useful for debugging!

Shared Memory

What is Shared Memory?

Shared memory is the fastest IPC mechanism available. Instead of copying data between processes through the kernel (as pipes and message queues do), shared memory allows multiple processes to directly access the same region of physical memory.

Think of it like a shared whiteboard in an office. Instead of writing notes on paper and passing them to each other (pipes), two people can both look at and write on the same whiteboard. No copying needed — both see changes immediately.

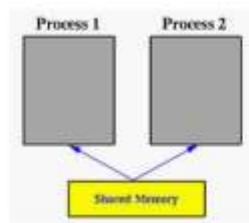


Figure 3: Shared memory concept – Process 1 and Process 2 each have their own address space (gray). The yellow shared memory region is mapped into both address spaces, so both processes can read and write it directly.

OS CONCEPT

With pipes and message queues, data is copied TWICE: from the sender's address space into a kernel buffer, and then from the kernel buffer into the receiver's address space. Shared memory eliminates BOTH copies — the kernel maps the same physical memory pages into both processes' virtual address spaces using the page table. The downside is that you must handle synchronization yourself (using semaphores or mutexes) to prevent race conditions.

How Shared Memory Works (Step by Step)

Using shared memory involves three steps for each process:

Step	Server (Writer)	Client (Reader)
1. Create / Access	shmget(key, size, IPC_CREAT 0666) — creates the segment	shmget(key, size, 0666) — accesses existing segment
2. Attach	shmat(shmid, NULL, 0) — maps segment into address space	shmat(shmid, NULL, 0) — maps segment into address space
3. Use	Read/write the memory using the pointer from shmat()	Read the memory using the pointer from shmat()
4. Detach	shmdt(ptr) — unmaps the segment from address space	shmdt(ptr) — unmaps the segment from address space
5. Remove	shmctl(shmid, IPC_RMID, NULL) — deletes the segment	(server is responsible for cleanup)

System V Shared Memory API

System Call	Header	Purpose
shmget(key, size, flags)	<sys/shm.h>	Create or access a shared memory segment (returns segment ID)
shmat(shmid, NULL, 0)	<sys/shm.h>	Attach the segment to the process address space (returns pointer)
shmdt(ptr)	<sys/shm.h>	Detach the segment from the process address space
shmctl(shmid, cmd, buf)	<sys/shm.h>	Control: IPC_RMID to remove, IPC_STAT to get info
ftok(path, proj_id)	<sys/ipc.h>	Generate a unique IPC key (same as for message queues)

Example 3: Shared Memory Server (Writer)

The server creates a shared memory segment, attaches it, writes data from user input, and keeps the segment alive for the client to read.

```
shm_server.c
```

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <unistd.h>
#include <sys/shm.h>    // shmget(), shmat(), shmdt(), shmctl()
#include <string.h>

int main() {
    void *shared_memory;
    char buff[100];
    int shmid;

    // Step 1: Create shared memory segment
    // Key: 1122, Size: 1024 bytes, Permissions: 0666 (rw for all)
    shmid = shmget((key_t)1122, 1024, 0666 | IPC_CREAT);
    if (shmid == -1) { perror("shmget"); exit(1); }
    printf("Key of shared memory is %d\n", shmid);

    // Step 2: Attach – map the segment into this process's address space
    shared_memory = shmat(shmid, NULL, 0);
    if (shared_memory == (void *)-1) { perror("shmat"); exit(1); }
    printf("Process attached at %p\n", shared_memory);

    // Step 3: Write data into shared memory
    printf("Enter some data to write to shared memory:\n");
    read(0, buff, 100);           // Read from stdin (FD 0)
    strcpy(shared_memory, buff);  // Copy data into shared memory
    printf("You wrote: %s\n", (char *)shared_memory);

    // Note: We do NOT detach or remove here so the client can read it
    // In production code, you would coordinate with the client
    return 0;
}

```

Example 4: Shared Memory Client (Reader)

shm_client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    void *shared_memory;
    int shmid;

    // Step 1: Access the EXISTING shared memory (same key: 1122)
    // No IPC_CREAT – the server must create it first

```

```

shmid = shmget((key_t)1122, 1024, 0666);
if (shmid == -1) { perror("shmget"); exit(1); }
printf("Key of shared memory is %d\n", shmid);

// Step 2: Attach – map the same segment into THIS process
shared_memory = shmat(shmid, NULL, 0);
if (shared_memory == (void *)-1) { perror("shmat"); exit(1); }
printf("Process attached at %p\n", shared_memory);

// Step 3: Read directly from shared memory – NO copying!
printf("Data read from shared memory is: %s\n", (char *)shared_memory);

// Step 4: Detach and clean up
shmdt(shared_memory);
shmctl(shmid, IPC_RMID, NULL); // Remove the segment
return 0;
}

```

<pre> # Terminal 1 (Server) \$ gcc shm_server.c -o ser \$./ser Key of shared memory is 0 Process attached at 0x7108f3cff000 Enter some data to write: This is Example of shm You wrote: This is Example of shm </pre>	<pre> # Terminal 2 (Client) \$ gcc shm_client.c -o cli \$./cli Key of shared memory is 0 Process attached at 0x72b063a59000 Data read from shared memory is: This is Example of shm </pre>
--	---

```

yasir@DISDST-95783779:~/Lab-05$ gcc shm_server.c -o ser
yasir@DISDST-95783779:~/Lab-05$ gcc shm_client.c -o cli
yasir@DISDST-95783779:~/Lab-05$ ./ser
Key of shared memory is 0
Process attached at 0x7108f3cff000
Enter some data to write to shared memory
This is Example of shm
You wrote : This is Example of shm

yasir@DISDST-95783779:~/Lab-05$ ./cli
Key of shared memory is 0
Process attached at 0x72b063a59000
Data read from shared memory is : This is Example of shm

yasir@DISDST-95783779:~/Lab-05$ |

```

Figure 4: Shared memory in action – Server writes data (top), client reads the same data directly from shared memory (bottom). Notice the different attach addresses — same physical memory, different virtual addresses.

WARNING

Shared memory has NO built-in synchronization. If two processes write to the same location at the same time, you get a race condition — corrupted or unpredictable data. In production code, always use semaphores (covered in a future lab) to protect shared memory access. The examples here work correctly only because we manually coordinate: server writes first, then client reads.

TIP

Use `ipcs -m` to list all shared memory segments on the system. If your program crashes before calling `shmctl(IPC_RMID)`, the segment stays in the kernel. Remove orphaned segments with: `ipcrm -m <shmid>`.

Choosing the Right IPC Mechanism

Now that you have learned all major IPC mechanisms across Labs 06 and 07, here is a decision guide:

Scenario	Best IPC Method	Why
Parent sends data to child process	Unnamed Pipe (Lab 06)	Simplest, automatic sync, no setup
Two separate programs exchange small messages	Named Pipe / FIFO (Lab 06)	No need to be related, easy setup
Structured messages with type-based selection	Message Queue (this lab)	Message boundaries preserved, type filtering
Sharing large datasets between processes	Shared Memory (this lab)	Fastest — no data copying through kernel
Shell command piping (<code>ls grep</code>)	Unnamed Pipe + <code>dup2</code> (Lab 06)	Redirect stdout/stdin between commands
Notifying a process about an event	Signal (future lab)	Lightweight, asynchronous
Protecting shared resource from concurrent access	Semaphore (future lab)	Mutual exclusion for shared memory

Performance Comparison

Criteria	Unnamed Pipe	Named Pipe	Message Queue	Shared Memory
Data copies	2 (user→kernel→user)	2 copies	2 copies	0 copies (direct)
Setup complexity	Low	Low	Medium	Medium
Synchronization	Built-in	Built-in	Built-in	Manual (need semaphores)
Message boundaries	No (byte stream)	No (byte stream)	Yes	No (raw memory)
Persistence	No	File persists	Queue persists	Segment persists
Cleanup command	Automatic	<code>rm /path</code>	<code>ipcrm -q ID</code>	<code>ipcrm -m ID</code>

System Calls – Quick Reference

Message Queue System Calls

System Call	Header	Purpose
ftok(path, id)	<sys/ipc.h>	Generate unique IPC key from file path + project char
msgget(key, flags)	<sys/msg.h>	Create/open a message queue, returns queue ID
msgsnd(qid, msg, size, 0)	<sys/msg.h>	Send a message struct to the queue
msgrcv(qid, buf, size, type, 0)	<sys/msg.h>	Receive message of given type (0 = any)
msgctl(qid, IPC_RMID, NULL)	<sys/msg.h>	Delete the message queue

Shared Memory System Calls

System Call	Header	Purpose
shmget(key, size, flags)	<sys/shm.h>	Create/access shared memory segment, returns segment ID
shmat(shmid, NULL, 0)	<sys/shm.h>	Attach segment to process address space, returns pointer
shmdt(ptr)	<sys/shm.h>	Detach segment from process address space
shmctl(shmid, IPC_RMID, NULL)	<sys/shm.h>	Remove the shared memory segment from kernel

Useful Shell Commands

Command	Purpose
ipcs	List all IPC resources (queues, shared memory, semaphores)
ipcs -q	List only message queues
ipcs -m	List only shared memory segments
ipcrm -q <msgid>	Remove an orphaned message queue
ipcrm -m <shmid>	Remove an orphaned shared memory segment

Lab Tasks

Complete the following tasks. Document your source code, compilation commands, and output screenshots.

Task 1: Message Queue – Student Record System

1. Define a message struct with fields: mtype (long), roll_number (int), name (char[50]), and cgpa (float).
2. Write a sender program that takes student records from user input and sends them to a message queue.
3. Write a receiver program that reads and displays all student records from the queue in a formatted table.

Task 2: Message Queue with Priority

1. Write a sender that sends 5 messages: two with mtype=1 (normal), two with mtype=2 (important), and one with mtype=3 (urgent).
2. Write a receiver that first reads all type-3 messages, then type-2, then type-1 (priority ordering).
3. Observe: even though messages were sent in mixed order, you receive them grouped by priority. Explain how mtype filtering enables this.

Task 3: Shared Memory – Chat Application

1. Create a shared memory segment that holds a structure with: a message buffer (char[256]) and a flag (int) indicating if a new message is ready.
2. Write a writer program that reads user input and writes it to shared memory, setting the flag to 1.
3. Write a reader program that polls the flag in a loop (with a small sleep). When the flag is 1, it reads and prints the message, then resets the flag to 0.
4. Discuss: why is this polling approach inefficient? What would be a better solution? (Hint: semaphores)

Task 4: Shared Memory Counter – Race Condition Demo

1. Create a shared memory segment containing a single integer counter initialized to 0.
2. Fork two child processes. Each child increments the counter 100,000 times in a loop.
3. After both children finish (use wait()), the parent reads and prints the counter.
4. Run the program 5 times. Is the result always 200,000? If not, explain why (race condition). What IPC mechanism would fix this?

Task 5: IPC Cleanup Script (Bonus)

1. Write a shell script called `ipc_cleanup.sh` that lists all message queues and shared memory segments owned by the current user.
2. The script should ask for confirmation, then remove all orphaned IPC resources using `ipcrm`.
3. This is a practical tool you will use throughout the rest of this course whenever your programs crash and leave IPC resources behind.

— End of Lab 07 —